S = Simple, M = Medium, D = Difficult

1-3 S/M/D Expression without sequence and strings [1m each]

1	The following code is in a standalone Python file. What is the output of the code when it is run? `print(10 + 20 // 3)`	**16** 16.0 16.666666666666668 17 10
2	The following code is in a standalone Python file. What is the output of the code when it is run? `print(bool(bool(False + 1) - 1))`	True **False** None 0 Running the code results in an error
3	The following code is in a standalone Python file. What is the output of the code when it is run? `print(5**3 or None and not 85 % 10 / 2)`	**125** True False None 2.5

4-6 S/M/D String manipulations [1m each]

4	The following code is in a standalone Python file. What is the output of the code when it is run? `print('recursion'[2:7][2])`	**r** c s u i

5	The following code is in a standalone Python file. What is the output of the code when it is run? `print('never'[::-1] + 'give'[::3] + 'up'[-3:-2:-1])`	**`revenge`** `revengeu` `revengiv` `revengivu` Running the code results in an error
6	The following code is in a standalone Python file. What is the output of the code when it is run? `print('_'.join('nus soc'.upper().split()))`	**`NUS_SOC`** `NUS_ _SOC` `N_U_S_S_O_C` `N_U_S_ _S_O_C` `NUS SOC`

7-8 S/D List manipulations [1m each]

7	The following code is in a standalone Python file. What is the output of the code when it is run? `lst = [1, 1, 2, 3, 5, 8]` `lst.pop()` `lst.insert(3, 4)` `print(lst)`	**`[1, 1, 2, 4, 3, 5]`** `[1, 2, 3, 4, 5, 8]` `[1, 1, 2, 3, 3, 5]` `[1, 2, 3, 5, 3, 8]` `[1, 1, 2, 3, 5, [3, 4]]`

8	The following code is in a standalone Python file. What is the output of the code when it is run? `x = [1, 9]` `y = [x, x[:], x.copy()]` `z = [y, y]` `y[1][1] = 7` `z[0][0][0] = 0` `print(z[1])`	**`[[0, 9], [1, 7], [1, 9]]`** `[[0, 7], [0, 7], [1, 9]]` `[[0, 9], [1, 7], [0, 9]]` `[[0, 7], [0, 7], [0, 7]]` `[[0, 7], [1, 9], [1, 9]]`

9-10 S/M Tuple manipulations [1m each]

9	The following code is in a standalone Python file. What is the output of the code when it is run? `print((1,) * 3 + (5,7))`	**`(1, 1, 1, 5, 7)`** `(3, 5, 7)` `(1, 1, 1, (5, 7))` `(3, (5, 7))` Running the code results in an error
10	The following code is in a standalone Python file. What is the output of the code when it is run? `a = ((1), (2,3), (4,5,6))` `print(a[0][0] + a[1][1] + a[2][2])`	`10` `9` `8` `(1, 3, 6)` **Running the code results in an error** **NOTE** Since `a[0]` is not a tuple, accessing `a[0][0]` from the integer value will produce a TypeError, akin to trying to compute `1[0].(1) != (1,)`

11-12 S/M Set [1m each]

11	The following code is in a standalone Python file. What is the output of the code when it is run? `s1 = {'woody', 'buzz', 'bopeep', 'hamm'}` `s2 = {'rex', 'hamm', 'buzz'}` `s3 = {'woody', 'slinky'}` `print(s1 & s2 | s1 ^ s3)`	**`{'bopeep', 'hamm', 'buzz', 'slinky'}`** `{'woody', 'hamm', 'bopeep', 'buzz', 'slinky'}` `{'rex', 'woody', 'bopeep'}` `{'woody', 'bopeep', 'slinky'}` `{'buzz', 'woody', 'hamm'}`

12	Which of the following expressions will cause an error when evaluated?	**`({10} ^ {2*5}).pop()`** `set('league') - set('legends')` `set('house').remove(min(set('mouse')))` `set([1,1,2,3,5]).add(8)` None of the above **NOTES** `^` is an XOR function. Given sets `A` and `B`, `A ^ B` will produce a set of all elements that are in either `A` or `B`, but not both. `{10} ^ {2*5}` evaluates to an empty set since `2*5=10`, making both sets have the same sole element. Following this, an attempt to use `pop()` on the empty set will end in a TypeError. Other options compute to: `{'a', 'u'}` `{'h', 'o', 'u', 's'}` `{1, 2, 3, 5, 8}`

13-15 S/M/D Dict [1m each]

13	The following code is in a standalone Python file. What is the output of the code when it is run? `print({3:'T', 4:'F', 5:'F', 6:'S', 7:'S', 8:'E', 9:'N'}[4])`	**`F`** `T` `S` `E` `N`

14	The following code is in a standalone Python file. What is the output of the code when it is run? `codes = {'charlie':3, 'victor':22, 'juliet':10, 'oscar':15, 'romeo':18}` `del codes['oscar']` `print(codes.get('romeo',10) + codes.get('juliet',22) + codes.get('oscar',3))` (For this question the last line was printed "print(codes. get" in exemplify)	**31** 28 35 43 Running the code results in an error
15	The following code is in a standalone Python file. What is the output of the code when it is run? `d = {(0,1): [9, 15, 24],` `(1,0): [16, 24, 40],` `0: (25, 55),` `1: (31, 63)}` `print(list(d)[1][0])`	**1** 31 16 [16, 24, 40] Running the code results in an error

16-17 S/M Selection (if-then-else) [1m each]

16	The following code is in a standalone Python file. What is the output of the code when it is run? `picture, paint, perfect = 1, 0, 0` `if picture:` `    words = 1000` `    if paint:` `        print(words)` `else:` `    words = 100` `    if not perfect:` `        words -= 10` `    print(words)`	**Empty (Nothing is printed)** 1000 100 90 10
17	The following code is in a standalone Python file. What is the output of the code when it is run? `def foo(L, x, flag):` `    k = 0` `    if flag and x > -1:` `        k = x` `    elif x % 2:` `        k = 1 - x` `    if k not in L:` `        return [k, x]` `    if L:` `        return L[1:]` `    return L` `print(foo([0, 19, 29], 19, 0))`	**[-18, 19]** [0, 19] [19, 29] [19, 19] []

18-20 S/M/D Repetitions (loops) [1m each]

18	Suppose we have a loop structure as follows: `while x < 100:` `  print(x)  # our print statement` `  x += x` If `x = 25`, how many times will the value of `x` be printed? (x = 25 means that x = 25 before the code executed)	**2 times** 3 times 1 time 4 times Nothing is printed out
19	The following code is in a standalone Python file. What is the output of the code when it is run? `base_stats = {"hp": 78, "atk": 107, "def": 75,` `  "spa": 100, "spd": 100, "speed": 70}` `total = 0` `for stat in base_stats:` `  total += base_stats.get(stat, 0)` `print(total)`	**530** 520 540 510 500
20	The following code is in a standalone Python file. What is the output of the code when it is run? `stops = ["port_klang", "klang", "shah_alam",` `  "subang_jaya", "petaling_jaya", "kuala_lumpur"]` `pswd = ""` `i = 1` `for s in stops:` `  pswd += s[i % 2]` `  i += 1` `print(pswd)`	**okhsek** pksspk plsupu rlautu Running the code results in an error

21-22 S/M Recursion [1m each]

21	The following code is in a standalone Python file. What is the output of the code when it is run? `def get_bmis(lst):` `  if len(lst) < 1:` `  return lst` `  return get_bmis(lst[:-1]) + [lst[-1][0]//lst[-1][-1]**2]` `patients = [(65,1.6), (95,1.8)]` `print(get_bmis(patients))`	**`[25.0, 29.0]`** `[25.390624999999996, 29.320987654320987]` `[40.0, 52.0]` `[40.625, 52.77777777777778]` Running the code results in an error
22	The following code is in a standalone Python file. What is the output of the code when it is run? `def magic(lst):` `  return [] if not lst \` `  else lst if len(lst) == 1 \` `  else ['T', *magic(lst[1:])] if lst[0]%2 == 0 \` `  else ['O', *magic(lst[1:])]` `print(magic([1,9,8,3,2,4,9]))`	**`['O', 'O', 'T', 'O', 'T', 'T', 9]`** `['O', 'O', 'T', 'O', 'T', 'T', 'O']` `['O', 9, 8, 3, 0, 4, 9]` `[] # empty list` Running the code results in an error

23-24 S/M Lambda [1m each]

23	The following code is in a standalone Python file. What is the output of the code when it is run? `print((lambda x: x**2 + x + 1)(5))`	**`31`** `25` `16` `6` `5`

24	The following code is in a standalone Python file. What is the output of the code when it is run? `def foo(f, x):` `  return f(x*2) + f(x)` `print(foo(lambda a: a if a%2==0 else a+1, 7))`	**22** 24 21 8 7

25-26 S/M OOP [1m each]

25	The following code is in a standalone Python file. What is the output of the code when it is run? `class Node:` `  def __init__(self, val=0, next=None):` `  self.val = val` `  self.next = next` `n1 = Node(5**2)` `n1.next = Node(4**2)` `print(n1.val + n1.next.val)`	**41** 25 16 9 Running the code results in an error
26	The following code is in a standalone Python file. What is the output of the code when it is run? `class Coords:` `  def __init__(self, x, y):` `  self.x = x` `  self.y = y` `def manhattan_dist(c1,c2):` `  return abs(c2.x-c1.x) + abs(c2.y-c1.y)` `print(manhattan_dist(Coords(4,2),Coords(2,5)))`	**5** 3 1 9 Running the code results in an error

27-28 S/D OOP (Inheritance) [1m each]

27	The following code is in a standalone Python file. What is the output of the code when it is run? `class Zombie:` `  def __init__(self):` `  self._hp = 100` `  self.toughness = 'Unremarkable'` `  def get_hp(self):` `  return self._hp` `class ConeheadZombie(Zombie):` `  def __init__(self):` `  super().__init__()` `  self.toughness = 'Medium'` `  self._defense_hp = 170` `  def get_hp(self):` `  return self._hp + self._defense_hp` `tugboat = ConeheadZombie()` `print(tugboat.get_hp())`	**270** 100 170 Running the code results in an error None of the above

28	The following code is in a standalone Python file. What is the output of the code when it is run? `class Item:` `  def __init__(self, name='', price=0):` `  self.name = name` `  self.price = max(0, price)` `  def __str__(self):` `  return f'Item name: {self.name}\n' \` `  + f'Item price: ${self.price}'` `class StoreItem(Item):` `  def __init__(self, name='', price=0, profit_rate=0):` `  super().__init__(name, price)` `  self.pr = min(max(0, profit_rate), 1)` `  def __str__(self):` `  nett_price = self.price * (1+self.pr)` `  return f'Item name: {self.name} (Nett Price)\n' \` `  + f'Item price: ${nett_price}'` `mug = StoreItem("IKEA Mug", 2, 0.1)` `print(mug)`	**`Item name: IKEA Mug (Nett Price)`** **`Item price: $2.2`** `Item name: IKEA Mug` `Item price: $2.2` `Item name: IKEA Mug (Nett Price)` `Item price: $2` `Item name: IKEA Mug` `Item price: $2` Running the code results in an error **NOTES** The `__str__()` dunder method is similar to something like the `toString()` method in Java and C++. Overriding this method in any class allows one to prepare an output string for any attempt to merely print the object, typically with useful/actionable information to people instead of a generic message like `<__main__.StoreItem object at 0x(address)>` (i.e., surface level data about the created object).

29-30 Special Python Packages [1m each]

29	What is the full name of the python package “PIL”?	**`Python Imaging Library`** `Python Installation Library` `Python IDLE Library` `Python Itertools Library` None of the above
30	The following code is in a standalone Python file. What is the output of the code when it is run? `import numpy as np` `a = np.array([1,2,3,4])` `print(a > 3)`	**`[False False False  True]`** `[4]` `4` `1` Running the code results in an error **NOTE** There should be two space characters as padding before `True`. This is a numpy quirk to some number values as well. The quirk allows for the space character and the `True` keyword to take up the same length as just `False` in a numpy array, and for non-negative values the same length as negative values or smaller values the same length as the larger values (e.g., `1` and `11` in the same `numpy` array).

31-32 S/M/D Long – Functions [2m each]

<table>
<tr>
<td>31</td>
<td>The following code is in a standalone Python file. What is the output of the code when it is run?


```
def feed(n):
    return n+1

def beef():
    return n*2

def babe():
    n = 5
    return feed(beef())

n = 10
print(babe())
```

</td>
<td>21
11
10
12
6</td>
</tr>
<tr>
<td>32</td>
<td>The following code is in a standalone Python file.


```
def merry(c):
    return 2 if c in 'aeiou' else 1

def eat(s, r):
    if not s:
        return ''
    return 'nom'*r + drink(s[1:], merry(s[0]))

def drink(s, r):
    if not s:
        return ''
    return 'yum'*r + eat(s[1:], merry(s[0]))

print(eat('noodles', 1))
```


When the code is run, how many times will the substring <code>'yum'</code> appear in the output?</td>
<td>4
3
5
6
7</td>
</tr>
</table>

33	The following code is in a standalone Python file. What is the output of the code when it is run? `def two(x):` `  one(x)` `def one(x):` `  print(x*2)` `def one(x):` `  print(x*3)` `two(10)`	**30** `20` Empty (Nothing is printed) Running the code results in an error, because the definition of `one()` comes after the definition of `two()` that contains the call to `one()` Running the code results in an error, because `one()` is defined more than once

34-35 S/D Long – Searching and Sorting [2m each]

34	The following code is in a standalone Python file. What is the output of the code when it is run? `def mysearch(x, L):` `  myL = sorted(L)` `  def seek(low, high):` `  if low > high:` `  return -1` `  mid = (low + high) // 2` `  if x == myL[mid]:` `  return mid` `  elif x < myL[mid]:` `  return seek(low, mid-1)` `  else:` `  return seek(mid+1, high)` `  return seek(0, len(myL)-1)` `L = list('hexagonal')` `print(mysearch('g', L))`	**3** `4` `2` `-1` `'g'`

35	The following code is in a standalone Python file. What is the output of the code when it is run? `def mysort(L):` `  myL = L.copy()` `  changed = True` `  while changed:` `  changed = False` `  for i in range(1,len(myL)):` `  if myL[i-1] < myL[i]:` `  myL[i-1], myL[i] = myL[i], myL[i-1]` `  changed = True` `  return myL` `L = [11, 35, 48, 29, 5, 17, 20, 8]` `print(mysort(L))`	**[48, 35, 29, 20, 17, 11, 8, 5]** [5, 8, 11, 17, 20, 29, 35, 48] [8, 20, 17, 5, 29, 48, 35, 11] [11, 35, 48, 29, 5, 17, 20, 8] [11, 48, 35, 29, 20, 17, 8, 5]

36-37 S/D Long – Multi-dimensional Array [2m each]

36	The following code is in a standalone Python file. What is the output of the code when it is run? `def fmat(m):` `  r = len(m)` `  c = len(m[0])` `  mo = []` `  for i in range(r):` `  mo.append(m[i][::-1])` `  return mo` `m = [[2, 4, 6, 8],` `  [3, 6, 9, 12],` `  [5, 10, 15, 20]]` `print(fmat(m))`	**`[[8, 6, 4, 2],`** **`[12, 9, 6, 3],`** **`[20, 15, 10, 5]]`** `[[5, 10, 15, 20],` `[3, 6, 9, 12],` `[2, 4, 6, 8]]` `[[20, 15, 10, 5],` `[12, 9, 6, 3],` `[8, 6, 4, 2]]` `[[2, 3, 5],` `[4, 6, 10],` `[6, 9, 15],` `[8, 12, 20]]` `[[8, 12, 20],` `[6, 9, 15],` `[4, 6, 10],` `[2, 3, 5]]`

37	The following code is in a standalone Python file. What is the output of the code when it is run? `def mul(n, L):` `  return [n*x for x in L]` `def supermul(L, M):` `  if len(L) != len(M):` `  return []` `  return [ mul(L[i], M[i]) for i in range(len(L)) ]` `m1 = [[1,2,3],` `  [2,3,4],` `  [3,4,5]]` `m2 = [[1,0,1],` `  [0,1,1],` `  [1,1,0]]` `print(supermul(list(map(max,m1)), m2))`	**`[[3, 0, 3], [0, 4, 4], [5, 5, 0]]`** `[[1, 0, 3], [0, 3, 4], [3, 4, 0]]` `[[1,0,1], [1,0,1], [1,0,1],` ` [0,1,1], [0,1,1], [0,1,1],` `[0,1,1],` ` [1,1,0], [1,1,0], [1,1,0],` `[1,1,0], [1,1,0]]` `[[], [], []]` `[]`

38-39 S/D Long – Program Tracing [2m each]

<table>
<tr>
<td>38</td>
<td>The following code is in a standalone Python file.


```
try:
    x = int(input('Enter x: '))
    y = int(input('Enter y: '))
    z = x/y
except ValueError:
    print('Alamak')
except ZeroDivisionError:
    raise NameError('Oopsies')
else:
    print(z)
finally:
    print('Phew')
```


Which of the following is a possible output when we run the code?</td>
<td>Enter x: a
Alamak
Phew

Enter x: 4
Enter y: 0
Oopsies
Phew

Enter x: 6
Enter y: 5
1.2

All of the above

None of the above</td>
</tr>
<tr>
<td>39</td>
<td>The following code is in a standalone Python file. What is the output of the code when it is run?


```
d1 = {1:'give', 2:'you', 3:'up'}
d2 = {1:'let', 2:'you', 3:'down'}
d3 = {1:'make', 2:'you', 3:'cry'}

never = {1:d1, 2:d2, 3:d3}
gonna = never.copy()
rick = {1:never, 2:gonna, 3:never}

gonna.update([(1, {1:'tell', 2:'a', 3:'lie'})])
gonna[3].update([(1,'say'), (2,'good'), (3,'bye')])

print(rick[1][1][1], rick[2][2][2], rick[3][3][3])
```

</td>
<td>give you bye
give you cry
tell you cry
tell you bye
Running the code results in an error</td>
</tr>
</table>

40-49 FITB [5m each]

#	QUESTION	MODEL SOLUTION
40	Write a function `repeat(s,n)` that will take in a string `s` and repeat every character of the string `n` times. Here are some sample outputs: `>>> print(repeat('abc',4))` `aaaabbbbcccc` `>>> print(repeat('IT5001',3))` `IIITTT555000000111` Fill in the blank(s) in the following code to complete the function: `def repeat(s,n):` `  output = ''` `  for c in ___BLANK_1___:` `  output += ___BLANK_2___` `  return output`	`def repeat(s,n):` `  output = ''` `  for c in s:` `  output += c*n` `  return output`

41	An acronym is an abbreviation formed using the initial letters of a multi-word name or phrase. Acronyms are often spelled with the initial letter of each word in all caps with no punctuation. Given a string with only alphabets and space, write a function to return its acronym in upper case. Here are some sample outputs: `>>> print(acronym('Ministry of Education'))` `MOE` `>>> print(acronym('Primary School Leaving Examination'))` `PSLE` `>>> print(acronym('self contained underwater breathing apparatus'))` `SCUBA` Fill in the blank(s) in the following code to complete the function: `def acronym(s):` `  output = ''` `  for w in ___BLANK_1___ :` `  output += ___BLANK_2___` `  return ___BLANK_3___`	`def acronym(s):` `  output = ''` `  for w in s.split():` `  output += w[0]` `  return output.upper()`

42

In this question and the next (Q42 and Q43), you need to complete two related functions.
For this question, write a function that takes in an integer `n` > 0 and returns the product of all its digits, e.g.:

697 → 6 x 9 x 8 = 378

We call this the “multiplying digit” function, `md`. Here are some sample outputs:

```
>>> print(md(697))
378
>>> print(md(378))
168
>>> print(md(168))
48
>>> print(md(48))
32
>>> print(md(32))
6
```

Fill in the blank(s) in the following code to complete the function:

```
def md(n):
    output = 1
    while ___BLANK_1___:
        output *= ___BLANK_2___
        n = ___BLANK_3___
    return output
```

```
def md(n):
    output = 1
    while n > 0:
        output *= n%10
        n = n // 10
    return output
```

43

This is a continuation of the previous question and you can assume the function `md(n)` is well implemented. Namely, you can call the function `md()` in your code.

If we keep applying the above operation to a number, it will finally reach a single digit number. E.g.:

$$697 \rightarrow 378 \rightarrow 168 \rightarrow 48 \rightarrow 32 \rightarrow 6$$

It took 5 steps to convert 697 to a single digit. We say that the “persistence” of 697 is 5.

Write a function that takes in an integer `n` > 0 and returns the persistence of `n`. Here are some sample outputs:

```
>>> print(persistent(697))
5
>>> print(persistent(277777788888899))
11
```

Fill in the blank(s) in the following code to complete the function:

```
def persistent(n):
    p = ___BLANK_1___
    while ___BLANK_2___:
        n = ___BLANK_3___
        p = ___BLANK_4___
    return p
```

```
def persistent(n):
    p = 0
    while n > 9:
        n = md(n)
        p = p + 1
    return p
```

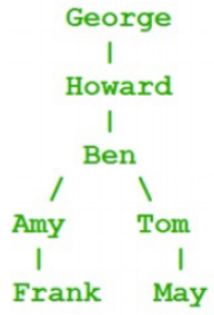
44	Grouchy Smurf doesn't like a lot of things. Every day, he "hates" a certain alphabet and he doesn't want to see it. Write a function `grouchy(c)` that will produce a lambda function to remove all occurrences of the character `c` in its input. Here are some sample outputs: `>>> no_a = grouchy('a')` `>>> print(no_a('banana'))` `bnn` `>>> print(no_a('autographically'))` `utogrphiclly` `>>> no_o = grouchy('o')` `>>> print(no_o('good afternoon'))` `gd afternn` `>>> print(no_o('sociobiological'))` `scibilgical` Fill in the blank(s) in the following code to complete the function: `def grouchy(c):` `  return lambda x: ___BLANK_1___`	`def grouchy(c):` `  return lambda x:''.join(filter(lambda w:w!=c,x))`

45	Given a deep list that only contains lists or integers, we want to add one more layer of list to every single integer. We call this function `dab` (deep add bracket). Here are some sample outputs: `>>> print(dab([[1,2],3,[4]]))` `[[[1], [2]], [3], [[4]]]` `>>> print(dab([1,2,[[3,4],[5,6,[7],8],9]]))` `[[1], [2], [[[3], [4]], [[5], [6], [[7]], [8]], [9]]]` Fill in the blank(s) in the following code to complete the function: `def dab(lst):` `  if not lst:` `  return ___BLANK_1___` `  if type(lst)!=list:` `  return ___BLANK_2___` `  return ___BLANK_3___`	`def dab(lst):` `  if not lst:` `  return **[]**` `  if type(lst)!=list:` `  return **[lst]**` `  return **[dab(lst[0])] +`** **`dab(lst[1:])`**

46

Ancestry Trees Height

This question is an add-on to our Assignment 5. Remember that we use the `parent` dictionary to represent a family tree. E.g., the following family tree:



is represented by the following dictionary:

```
parent = {'Amy': 'Ben', 'Tom': 'Ben', 'Frank': 'Amy',
'May': 'Tom', 'Ben': 'Howard', 'Howard': 'George'}
```

Write a function to compute the number of generations in the family tree. For the above tree, the number of generations is 5 because there are 5 layers. But if May has a child Carl, then the number of generations becomes 6. Here are some sample outputs:

```
>>> parent = {'Amy': 'Ben', 'Tom': 'Ben', 'Frank': 'Amy', 'May':
'Tom', 'Ben': 'Howard', 'Howard': 'George'}

>>> print(n_gen(parent))
5
>>> parent['Carl']='May'
>>> print(n_gen(parent))
6
```

Fill in the blank(s) in the following code to complete the function:

```
def n_gen(parent):
    ans = 0
    for child in ___BLANK_1___:
        n_g = 1
        while ___BLANK_2___:
```

```
def n_gen(parent):
    ans = 0
    for child in parent.keys():
        n_g = 1
        while child in
parent.keys():
            child = parent[child]
            n_g = n_g + 1
        ans = max(ans,n_g)
    return ans
```

	`` child = ___BLANK3___ n_g = ___BLANK_4___ ans = max(ans,n_g) return ans ```	
47	Given a 2D array that contains only integers 0 or 1, write a function to return a matrix where all 0 values are changed to 1, and vice versa. Here is an example: ``` >>> m = [[0,0,1,1,0,0,1,1,0,0],[1,1,1,1,1,1,1,1,1,1], [1,1,1,1,1,1,1,1,1,1],[0,0,1,1,0,0,1,1,0,0]] >>> pprint(m) [[0, 0, 1, 1, 0, 0, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [0, 0, 1, 1, 0, 0, 1, 1, 0, 0]] >>> pprint(invert(m)) [[1, 1, 0, 0, 1, 1, 0, 0, 1, 1], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [1, 1, 0, 0, 1, 1, 0, 0, 1, 1]] ``` Fill in the blank(s) in the following code to complete the function: ``` def invert(m): r = len(m) c = len(m[0]) output = [] for ___BLANK_1___: new_row = [] for ___BLANK_2___: ___BLANK_3___ output.append(new_row) return output ```	``` def invert(m): r = len(m) c = len(m[0]) output = [] for row in m: new_row = [] for x in row: new_row.append(1-x) output.append(new_row) return output ```

48	Rewrite the function `invert(m)` in the previous question so that the function body consists of only one line. Your function cannot contain any semi-colon or recursion. Fill in the blank(s) in the following code to complete the function: `def invert(m):` `    return ___BLANK_1___`	`def invert(m):` `    return [[1-x for x in row] for row in m]`

49

Given **two** natural numbers **n and m**, find the **number of ways** in which the numbers that are **greater than or equal to m** can be added to get the **sum n**.

Examples:

***Input**: n = 3, m = 1*
***Output**: 3*
***Explanation:** Three different ways to get sum n such that each term is greater than or equal to m are 1 + 1 + 1, 1 + 2, and 3.*

***Input**: n = 2, m = 1*
***Output**: 2*
***Explanation:** Two different ways to get sum n such that each term is greater than or equal to m are 1 + 1 and 2.*

The following code actually works perfectly fine for this problem. However, there is a problem: the code is too slow.

```
def count_recur(m, n):
    # base case
    if n == 0:
        return 1
    if m > n or n < 0:
        return 0
    # include current m
    take = count_recur(m, n - m)
    # skip current m
    no_take = count_recur(m + 1, n)
    return take + no_take
```

Please add in a few lines to make this code runs a lot faster:

```
___BLANK_1___
def count_recur(m, n):
    if ___BLANK_2___:
        ___BLANK_3___
    # base case
    if n == 0:
        return 1
```

```
ans = dict()
def count_recur(m, n):
    if (m,n) in ans:
        return ans[(m,n)]
    # base case
    if n == 0:
        return 1
    if m > n or n < 0:
        return 0
    # include current m
    take = count_recur(m, n - m)
    # skip current m
    no_take = count_recur(m + 1, n)
    ans[(m,n)] = take+no_take
    return take + no_take
```

```
    if m > n or n < 0:
        return 0
    # include current m
    take = count_recur(m, n - m)
    # skip current m
    no_take = count_recur(m + 1, n)
    ___BLANK_4___
    return take + no_take
```

50 Free Response Question [2m]

Of the things you learned in this course, what captured your interest the most? If you could start the semester over, what advice would you give yourself for this class?